

CSE 215 • EXAM READY

Digital Logic Design

Complete Beginner Study Guide

0 1 0 0 1 1 0 0 0 1 0 0 1 1 1 1 0 1 0 0 0 1 1 1 0 1 0 0 1 0 0 1 0 1 0 0 0 0 1 1

Based on the uploaded lecture materials

Table of Contents



Introduction to Digital Systems

1.1 Analog vs. Digital

Before we talk about number systems, we need to understand the difference between two ways of representing information: **analog** and **digital**.

- **Analog data** can vary over a continuous range of values. Nothing "jumps" — it flows smoothly from one value to another. A classic example is a car speedometer: the needle can point to 42.3 km/h, 42.31 km/h, or any value in between.
- **Digital quantities** can only take on a fixed set of discrete (separate, countable) values — most commonly just two values: 0 and 1, also called "low" and "high". Examples include a digital computer, the decimal digits we write, and the letters of the alphabet.

WHY DOES THIS MATTER?

Digital circuits are built to reliably tell the difference between only two states (on/off). This simplicity is exactly what makes digital systems fast, accurate, and easy to design — which is the whole reason computers use binary instead of continuously varying signals.

1.2 What is a Digital System?

A **digital system** is a combination of devices designed to manipulate physical quantities or information that is represented in **digital form**. In simple words:

DEFINITION

A digital system is "a discrete information-processing system." The signals inside it carry *discrete* information (separate, distinct values) rather than continuously changing values.

1.3 Advantages of Digital Systems

Digital systems are used everywhere today because they offer several important benefits:

- **Greater accuracy or precision** — discrete values are less affected by small errors or noise.
- **Easier to design** — because the same basic building blocks (0s and 1s) are reused everywhere (this property is called generality).
- **Easier information storage** — digital data can be stored and retrieved reliably.
- **Programmability** — digital systems can be controlled by instructions (programs).
- **Speed** — digital circuits can process information extremely fast.
- **Economical** — mass production of digital components makes them cheap.

1.4 Limitation of Digital Technology

KEY LIMITATION

The real world is mainly **analog** — temperature, sound, light, and motion all vary continuously. A digital system cannot directly "understand" this continuous information.

1.5 Overcoming the Limitation

To let digital systems work with real-world (analog) information, engineers follow a three-step process:

1. Convert the real-world analog input data into digital form.
2. Process this digital data using digital circuits.
3. Convert the result back into analog form, if needed (e.g., so we can hear it as sound or see it as a smooth image).

EXAM TIP

This 3-step process (Analog → Digital → Process → Analog) is a common short-answer question. Remember it as "convert in, process, convert out."

1.6 Digital Logic

The term **digital logic (design)** is used to describe the design and analysis of digital systems.

- Digital logic is concerned with the interconnection among digital components and modules.
- Digital logic design is a branch of engineering — and engineering, at its heart, means **problem solving**.

1.7 Why We Need Number Systems and Codes

Digital systems are built from circuits that process **binary digits** (0s and 1s). But very few real-life problems are naturally expressed in binary — we count in decimal, we write letters, we measure with all sorts of units.

So, a digital system designer must establish a **correspondence** (a mapping / relationship) between the binary digits processed by digital circuits and real-life numbers, events, and conditions. This is exactly what Chapter 2 onward will teach you: how to move between binary and the number systems we naturally use.

Number Systems

2.1 What is a Number System?

A **number system** is simply a way of writing numbers using a fixed set of symbols (digits), combined with rules for what each symbol's position means.

Base (Radix)

The **base** (also called the **radix**), written as r , tells us *how many different digit symbols* the number system uses.

REMEMBER

If the base is r , the digits (coefficients) used in that system range from 0 to $r - 1$. For example, base 10 uses digits 0 through 9 — ten symbols in total, which matches "base 10."

Digits

Digits are the individual symbols allowed in a number system. Decimal uses the symbols $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.

Positional Notation

In a positional number system, the **position** of a digit within the number is just as important as the digit itself. The same digit means a different amount depending on where it sits. For example, in the decimal number 7594, the digit 5 does not simply mean "five" — it means "five hundreds," because of its position.

Weight of Digits

Every position in a number carries a **weight**, which is the base raised to a power depending on the position. In decimal:

$$(7594)_{10} = (7 \times 10^3) + (5 \times 10^2) + (9 \times 10^1) + (4 \times 10^0)$$

In general, for a whole number with digits $a_n a_{n-1} \dots a_0$:

$$(a_n a_{n-1} \dots a_0)_{10} = (a_n \times 10^n) + (a_{n-1} \times 10^{n-1}) + \dots + (a_0 \times 10^0)$$

2.2 Binary (Base 2)

- **Base:** 2
- **Digits (bits):** 0, 1
- **Weights:** powers of 2 (... , 8, 4, 2, 1, then 1/2, 1/4, ...)

MEMORY TIP

Binary uses only 2 symbols because a digital switch can only be "on" or "off" — there is no in-between state, so 2 symbols is all we need.

2.3 Decimal (Base 10)

- **Base:** 10
- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- This is the number system we use in everyday life, most likely because humans have 10 fingers.

2.4 Octal (Base 8)

- **Base:** 8
- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7
- **Weights:** powers of 8

2.5 Hexadecimal (Base 16)

- **Base:** 16
- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- **Weights:** powers of 16

WHY DOES HEXADECIMAL USE A–F?

Base 16 needs 16 different digit symbols (0 to 15). But our ordinary numerals only go up to 9! So, to represent the values 10, 11, 12, 13, 14, and 15 using a *single* symbol each, we borrow letters: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

2.6 Binary / Octal / Decimal / Hexadecimal — Master Table

Table 2.1 — Equivalent values across the four number systems

Binary	Octal	Decimal	Hexadecimal
0000	0	0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6

0111	7	7	7
1000	10	8	8
1001	11	9	9
1010	12	10	A
1011	13	11	B
1100	14	12	C
1101	15	13	D
1110	16	14	E
1111	17	15	F

EXAM TIP

Memorising this table (or being able to rebuild it quickly) makes every conversion question in this guide much faster.

Exercises 2

Practice note: The lecture material does not include a separate numbered exercise for this introductory topic — instead, practise by re-deriving Table 2.1 from scratch (count in binary from 0000 to 1111, and write the matching octal, decimal and hex value for each row) until you can do it without looking.

Positional Value

3.1 How Place Value Works

As we saw in Chapter 2, every digit's contribution to a number depends on *where* it sits. This idea — called **positional value** or **place value** — is the foundation of every conversion technique in this guide.

Whole Numbers

For a decimal whole number with digits $a_n \ a_{n-1} \ \dots \ a_1 \ a_0$, the value is:

$$(a_n \ a_{n-1} \ \dots \ a_0)_{10} = (a_n \times 10^n) + (a_{n-1} \times 10^{n-1}) + \dots + (a_0 \times 10^0)$$

Worked Example Let's break down 7594 digit by digit.

$$\begin{aligned} 7594 &= 7 \times 10^3 + 5 \times 10^2 + 9 \times 10^1 + 4 \times 10^0 \\ &= 7 \times 1000 + 5 \times 100 + 9 \times 10 + 4 \times 1 \\ &= 7000 + 500 + 90 + 4 \\ &= 7594 \checkmark \end{aligned}$$

Fractional Values

Place value continues to the *right* of the decimal point too — except now the powers of the base become **negative** (i.e., fractions).

$$(2.75)_{10} = (2 \times 10^0) + (7 \times 10^{-1}) + (5 \times 10^{-2})$$

In general, for a number with both a whole part and a fractional part:

$$\begin{aligned} (a_n \ \dots \ a_0 . f_1 \ f_2 \ \dots \ f_m)_{10} \\ = (a_n \times 10^n) + \dots + (a_0 \times 10^0) + (f_1 \times 10^{-1}) + (f_2 \times 10^{-2}) + \dots + (f_m \times 10^{-m}) \end{aligned}$$

Worked Example Break down 2.75.

$$\begin{aligned} 2.75 &= 2 \times 10^0 + 7 \times 10^{-1} + 5 \times 10^{-2} \\ &= 2 + 0.7 + 0.05 \\ &= 2.75 \checkmark \end{aligned}$$

WHY?

Just like $10^1 = 10$ and $10^2 = 100$ (each step to the *left* multiplies by the base), each step to the *right* of the point *divides* by the base — that's exactly what a negative exponent means: $10^{-1} = 1/10$, $10^{-2} = 1/100$, and so on. This same idea works for *any* base, not just base 10, which is exactly what we use in the next chapter.

Exercises 3

Exercise 3(a): Add $(101101)_2$ with $(100111)_2$.

(This question tests place value indirectly through binary addition — full method taught in Chapter 12. Try expanding both numbers into their positional value form first as practice for this chapter.)

Solution 3(a)

```
Augend:   101101
Addend: + 100111
-----
Sum:      1010100
```

So $(101101)_2 + (100111)_2 = (1010100)_2$. The full column-by-column carry method is explained in Chapter 12.

Base-R to Decimal Conversion

4.1 The Formula

Converting *any* base into decimal always uses the same idea: multiply each digit by its positional weight (a power of the source base), then add everything together.

FORMULA

Value in decimal = $\sum (\text{digit} \times \text{source_base}^{\text{position}})$

Position numbering starts at 0 for the rightmost digit before the point, and counts upward to the left; positions after the point are negative, starting at -1 .

4.2 How Powers Work

Number the digit positions from right to left, starting at 0, for the integer part. Positions after the decimal (or "radix") point count downward starting at -1 .

4.3 Worked Examples

Example 1 — Binary to Decimal Convert $(1101.101)_2$ to decimal.

$$\begin{aligned}(1101.101)_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= 8 + 4 + 0 + 1 + 0.5 + 0 + 0.125 \\ &= (13.625)_{10}\end{aligned}$$

Row-by-row: the leftmost 1 sits in the $2^3 = 8$ column, the next 1 in the $2^2 = 4$ column, the 0 in the $2^1 = 2$ column (contributes nothing), the last 1 before the point is the $2^0 = 1$ column. After the point: the 1 is in the $2^{-1} = 0.5$ column, the 0 in the $2^{-2} = 0.25$ column (contributes nothing), and the final 1 is in the $2^{-3} = 0.125$ column.

Example 2 — Octal to Decimal Convert $(572.6)_8$ to decimal.

$$\begin{aligned}(572.6)_8 &= 5 \times 8^2 + 7 \times 8^1 + 2 \times 8^0 + 6 \times 8^{-1} \\ &= 5 \times 64 + 7 \times 8 + 2 \times 1 + 6 \times 0.125 \\ &= 320 + 56 + 2 + 0.75 \\ &= (378.75)_{10}\end{aligned}$$

Example 3 — Hexadecimal to Decimal

Convert $(2A.8)_{16}$ to decimal. Remember A = 10 in hexadecimal.

$$\begin{aligned}(2A.8)_{16} &= 2 \times 16^1 + 10 \times 16^0 + 8 \times 16^{-1} \\ &= 32 + 10 + 0.5 \\ &= (42.5)_{10}\end{aligned}$$

Example 4 — Base-5 to Decimal

Convert $(341.24)_5$ to decimal.

$$\begin{aligned}(341.24)_5 &= 3 \times 5^2 + 4 \times 5^1 + 1 \times 5^0 + 2 \times 5^{-1} + 4 \times 5^{-2} \\ &= 75 + 20 + 1 + 0.4 + 0.16 \\ &= (96.56)_{10}\end{aligned}$$

This example proves the formula works for *any* base — not just 2, 8, and 16.

COMMON MISTAKE

Forgetting that the position right after the point is **-1**, not 0 or 1. Students often mix up the direction of the exponent on the fractional side.

SHORTCUT

Write out the place-value headers (powers of the base) above each digit *before* multiplying — this avoids position-counting mistakes, especially for longer numbers.

Exercise 4(a)

Compute $(71)_8 - (56)_8$. (Subtraction of base- r values — full method in Chapter 13; use this to also practise reading base-8 digit values.)

Solution 4(a)

$$(71)_8 - (56)_8 = (13)_8$$

Exercise 4(b)

Compute $(21)_3 - (12)_3$.

Solution 4(b)

$$(21)_3 - (12)_3 = (2)_3$$

Decimal to Binary (and Any Base) Conversion

5.1 Two Methods Overview

Method	Works for
Sum-of-Weights Method	Binary only
Repeated Division-by-2 / Repeated Multiplication-by-2	Any base (Division-by-R / Multiplication-by-R)

5.2 Sum-of-Weights Method (Binary Only)

Determine the set of binary weights (powers of 2) whose sum equals the decimal number.

MEMORY TIP

Knowing the powers of 2 by heart (1, 2, 4, 8, 16, 32, 64, 128, 256, ...) makes this method very fast.

Worked Examples

$$\begin{array}{lll}
 (9)_{10} = 8 + 1 & = 2^3 + 2^0 & = (1001)_2 \\
 (18)_{10} = 16 + 2 & = 2^4 + 2^1 & = (10010)_2 \\
 (58)_{10} = 32 + 16 + 8 + 2 & = 2^5 + 2^4 + 2^3 + 2^1 & = (111010)_2 \\
 (0.625)_{10} = 0.5 + 0.125 & = 2^{-1} + 2^{-3} & = (0.101)_2
 \end{array}$$

5.3 Repeated Division-by-2 Method (For Whole Numbers)

To convert a **whole number** to binary: divide repeatedly by 2 until the quotient becomes 0. The **remainders** form the answer.

- The **quotient** is the result of the division — this is the number you keep dividing by 2 again.
- The **remainder** (0 or 1) is what's left over after each division — this becomes one bit of your answer.
- The *first* remainder produced is the **Least Significant Bit (LSB)** — the rightmost bit.
- The *last* remainder produced is the **Most Significant Bit (MSB)** — the leftmost bit.

WHY READ REMAINDERS BOTTOM TO TOP?

The very first division tells you whether the number is even or odd — that's the smallest, "least significant" piece of information, so it belongs on the right. The last division (when the quotient finally hits 0) tells you about the largest place value, so it belongs on the left. Reading bottom-to-top simply lines the bits up from most-significant to least-significant, left to right, the way we normally write numbers.

Worked Example — $(43)_{10}$ to Binary

2		43		
2		21	rem 1	← LSB
2		10	rem 1	
2		5	rem 0	
2		2	rem 1	
2		1	rem 0	
		0	rem 1	← MSB

Read remainders bottom to top: 1 0 1 0 1 1
 $(43)_{10} = (101011)_2$

5.4 Repeated Multiplication-by-2 Method (For Fractions)

To convert a **decimal fraction** to binary: multiply repeatedly by 2 until the fractional part becomes 0 (or until you have enough decimal places). The **carried digits** (the whole-number part produced at each step, called "carries") form the answer.

- The *first* carry produced is the **MSB** (leftmost bit after the point).
- The *last* carry produced is the **LSB** (rightmost bit).

Worked Example — $(0.3125)_{10}$ to Binary

	Carry	
$0.3125 \times 2 = 0.625$	0	← MSB
$0.625 \times 2 = 1.25$	1	
$0.25 \times 2 = 0.50$	0	
$0.5 \times 2 = 1.00$	1	← LSB

Read carries top to bottom: 0 1 0 1
 $(0.3125)_{10} = (.0101)_2$

INTEGER VS. FRACTION — KEY DIFFERENCE

Integer part	Fraction part
Division by base of target system	Multiplication by base of target system
Remainders are accumulated	Carries (whole-number parts) are accumulated
Read bottom to top → LSB to MSB order produced	Read top to bottom → MSB to LSB order produced

COMMON MISTAKE

Reading the division remainders top-to-bottom instead of bottom-to-top (or the multiplication carries the wrong way). Always double-check which end is the MSB.

SHORTCUT

Label the first and last remainder/carry as LSB/MSB immediately as you compute them — don't wait until the end to figure out the order.

Exercise 5(b)

Subtract $(100111)_2$ from $(101101)_2$. (A place-value / binary arithmetic check — full subtraction method in Chapter 13.)

Solution 5(b)

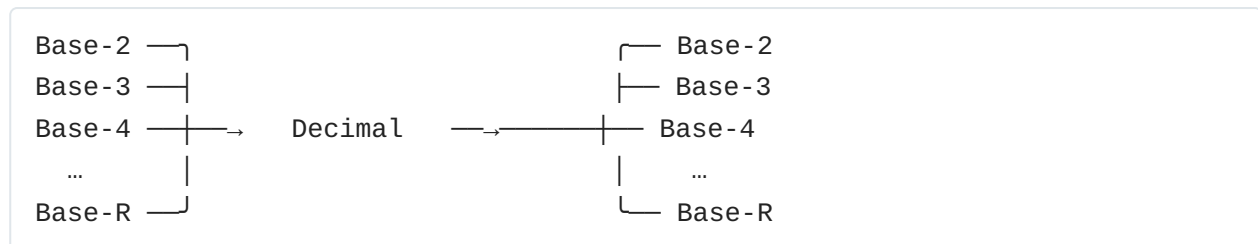
```
Minuend:      101101
Subtrahend: - 100111
-----
Difference:    000110
```

So $(101101)_2 - (100111)_2 = (000110)_2$.

Converting Between Any Two Bases

6.1 The General Rule

What if you need to convert, say, base-3 to base-7 directly? In general, conversion between any two non-decimal bases is done **via decimal** as a "bridge":



In other words:

1. Convert the source base into decimal (Chapter 4's method).
2. Convert that decimal value into the target base (Chapter 5's method, generalised to any base R).

WHY DECIMAL AS THE BRIDGE?

We already have one reliable method to go *into* decimal (multiply digits by positional weights) and one reliable method to go *out of* decimal (repeated division / multiplication by the target base). Decimal acts as a common "middle ground" so we only need to learn two general methods instead of a separate method for every possible pair of bases.

EXAM TIP

Some conversions have faster direct shortcuts that skip the decimal bridge entirely — binary ↔ octal and binary ↔ hexadecimal are the two most important ones. These are covered next, in Chapters 7 and 8.

Binary ↔ Octal Conversion

7.1 Why Grouping in 3 Bits Works

WHY 3 BITS?

Octal is base 8, and $8 = 2^3$. This means every single octal digit (0–7) can be represented *exactly* using 3 binary bits, with no overlap or waste. This is why binary and octal have such a clean, direct shortcut — unlike, say, binary to decimal, which needs the full positional-value formula.

7.2 Binary → Octal

Method: Partition (split) the binary number into groups of 3 bits, starting from the radix (decimal) point and working outward in both directions. Pad with zeros if a group is incomplete.

Worked Example

$$(10\ 111\ 011\ 001\ .\ 101\ 110)_2 = (2731.56)_8$$

Grouping:

10	111	011	001	.	101	110
2	7	3	1		5	6

Note on padding: the leftmost group here is only 2 bits ("10"); mentally pad it with a leading 0 to make "010" = 2 if it helps you double-check — the value doesn't change.

7.3 Octal → Binary (Reverse)

Method: Write each octal digit as its exact 3-bit binary equivalent, then join them together.

Worked Example

$$(2731.56)_8 = (10\ 111\ 011\ 001\ .\ 101\ 110)_2$$

7.4 More Worked Examples

Example

$$(1100100.10)_2 = (144.4)_8$$

SHORTCUT

Memorise the 3-bit patterns for octal digits 0–7 (000, 001, 010, 011, 100, 101, 110, 111) so you can convert instantly without recomputing.

COMMON MISTAKE

Grouping from the wrong end, or forgetting to pad the outermost group with zeros so every group has exactly 3 bits.

Exercise 1(a)

Convert $(10110001101011.111100000110)_2$ to octal.

Solution 1(a)

$$(10110001101011.111100000110)_2 = (26153.7406)_8$$

Exercise 2(a)

Convert $(673.124)_8$ to binary.

Solution 2(a)

$$(673.124)_8 = (110\ 111\ 011\ .\ 001\ 010\ 100)_2$$

Binary ↔ Hexadecimal Conversion

8.1 Why Grouping in 4 Bits Works

WHY 4 BITS?

Hexadecimal is base 16, and $16 = 2^4$. So every hexadecimal digit (0–F) is represented exactly by 4 binary bits — again, a perfect match with no waste.

8.2 Complete Binary–Hex Lookup Table

Table 8.1 — 4-bit binary groups and their hexadecimal digit

Binary	Hex	Binary	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

8.3 Binary → Hexadecimal

Method: Partition the binary number into groups of 4 bits, starting from the point and working outward, padding with zeros as needed.

Worked Example

$(101\ 1101\ 1001\ .\ 1011\ 1000)_2 = (5D9.B8)_{16}$

Grouping:

101 1101 1001 . 1011 1000
5 D 9 B 8

8.4 Hexadecimal → Binary (Reverse)

Worked Example

$$(5D9.B8)_{16} = (101\ 1101\ 1001\ .\ 1011\ 1000)_2$$

8.5 More Worked Examples

Example

$$(1100100.10)_2 = (64.8)_{16}$$

COMMON MISTAKE

Mixing up 3-bit grouping (octal) with 4-bit grouping (hex). Always double-check which target base you're converting to before deciding your group size.

Exercise 1(b)

Convert $(10110001101011.111100000110)_2$ to hexadecimal.

Solution 1(b)

$$(10110001101011.111100000110)_2 = (2C6B.F06)_{16}$$

Exercise 2(b)

Convert $(306.D)_{16}$ to binary.

Solution 2(b)

$$(306.D)_{16} = (0011\ 0000\ 0110\ .\ 1101)_2$$

Octal ↔ Decimal Conversion

9.1 Octal → Decimal: The Positional Method

This uses the exact same general Base-R-to-Decimal formula from Chapter 4, with the source base set to 8:

FORMULA

Decimal value = $\sum (\text{digit} \times 8^{\text{position}})$

Worked Example Convert $(572.6)_8$ to decimal.

$$\begin{aligned}(572.6)_8 &= 5 \times 8^2 + 7 \times 8^1 + 2 \times 8^0 + 6 \times 8^{-1} \\ &= 320 + 56 + 2 + 0.75 \\ &= (378.75)_{10}\end{aligned}$$

9.2 Decimal → Octal: Repeated Division / Multiplication by 8

This uses the general Decimal-to-Base-R method from Chapter 5, with the target base set to 8: divide the whole part repeatedly by 8 (reading remainders bottom to top), and multiply the fractional part repeatedly by 8 (reading carries top to bottom).

Worked Example — following the same style as $(43)_{10} \rightarrow \text{binary}$ in Chapter 5 Convert $(58)_{10}$ to octal using repeated division by 8.

$$\begin{array}{rcll} 8 & | & 58 & \\ 8 & | & 7 & \text{rem } 2 \leftarrow \text{LSB} \\ & & 0 & \text{rem } 7 \leftarrow \text{MSB} \end{array}$$

Read remainders bottom to top: 7 2
 $(58)_{10} = (72)_8$

Check (convert back using the positional method): $7 \times 8^1 + 2 \times 8^0 = 56 + 2 = 58 \checkmark$

EXAM TIP

Whenever you convert decimal to octal, always check your work by converting the octal answer back to decimal using the Chapter 4 formula — as shown above.

Exercise (based on Example 2 from Chapter 4)

Convert $(378.75)_{10}$ back to octal and verify it matches $(572.6)_8$.

Solution

Whole part: $378 \div 8$ repeatedly gives remainders that, read bottom to top, reproduce 572. Fractional part: $0.75 \times 8 = 6.00$, giving a single carry of 6. Combined: $(378.75)_{10} = (572.6)_8$ ✓ — matching the original example exactly, confirming the method works in both directions.

Hexadecimal ↔ Decimal Conversion

10.1 The Hex Digits A–F

Before converting, make sure these values are automatic for you:

Hex digit	A	B	C	D	E	F
Decimal value	10	11	12	13	14	15

10.2 Hexadecimal → Decimal: The Positional Method

FORMULA

Decimal value = $\sum (\text{digit} \times 16^{\text{position}})$

Worked Example Convert $(2A.8)_{16}$ to decimal.

$$\begin{aligned}
 (2A.8)_{16} &= 2 \times 16^1 + 10 \times 16^0 + 8 \times 16^{-1} \\
 &= 32 + 10 + 0.5 \\
 &= (42.5)_{10}
 \end{aligned}$$

10.3 Decimal → Hexadecimal: Repeated Division / Multiplication by 16

Worked Example Convert $(42)_{10}$ to hexadecimal.

$$\begin{array}{rcll}
 16 & | & 42 & \\
 16 & | & 2 & \text{rem } 10 \text{ (A)} \quad \leftarrow \text{LSB} \\
 & & 0 & \text{rem } 2 \quad \leftarrow \text{MSB}
 \end{array}$$

Read remainders bottom to top: 2 A

$$(42)_{10} = (2A)_{16}$$

Check: $2 \times 16^1 + 10 \times 16^0 = 32 + 10 = 42 \checkmark$

COMMON MISTAKE

Writing a remainder of 10–15 as a two-digit number ("10") instead of converting it to its single hex letter (A). Hexadecimal digits are always a single symbol.

Exercise

Convert $(378.75)_{10}$ to hexadecimal.

Solution

$378 \div 16$ repeatedly (reading remainders bottom to top) gives $17A_{16}$ for the whole part... actually let's verify directly using Chapter 8's known equivalence: since $(1100100.10)_2$ is not the same number, instead check directly: $378 = 23 \times 16 + 10$, $23 = 1 \times 16 + 7$, $1 = 0 \times 16 + 1 \rightarrow$ reading bottom to top: 1, 7, A $\rightarrow (17A)_{16}$. Fractional part: $0.75 \times 16 = 12.00 \rightarrow$ carry C. So $(378.75)_{10} = (17A.C)_{16}$.

Check: $1 \times 16^2 + 7 \times 16^1 + 10 \times 16^0 + 12 \times 16^{-1} = 256 + 112 + 10 + 0.75 = 378.75 \checkmark$

Octal ↔ Hexadecimal Conversion

There is no single direct grouping shortcut between octal and hexadecimal (unlike binary ↔ octal or binary ↔ hex), because 8 and 16 are not simple powers of one another in a way that lines up digit-for-digit. Instead, we combine methods you already know from earlier chapters.

Method 1 — Convert Through Binary

1. Convert the octal (or hex) number to binary, using the grouping method from Chapter 7 (octal, 3-bit groups) or Chapter 8 (hex, 4-bit groups).
2. Re-group that same binary number into groups of 4 (for hex) or 3 (for octal), using the method from the *other* chapter.

Worked Example — Octal to Hex via Binary Convert $(572.6)_8$ to hexadecimal.

Step 1 — Octal to binary (3-bit groups, Chapter 7):

```

5      7      2      .      6
101   111   010   .   110
(572.6)8 = (101111010.110)2
```

Step 2 — Regroup into 4-bit groups from the point outward (Chapter 8):

```

1 0111 1010 . 1100
pad → 0001 0111 1010 . 1100
   1      7      A      C
(101111010.110)2 = (17A.C)16
```

This matches the answer we found in Chapter 10 by going through decimal — confirming both routes agree.

Method 2 — Convert Through Decimal

1. Convert the source number to decimal using the Base-R-to-Decimal formula (Chapter 4 / 9 / 10).
2. Convert that decimal value into the target base using repeated division/multiplication (Chapter 5 / 9 / 10).

WHICH METHOD IS FASTER?

Method 1 (through binary) is usually faster and less error-prone, because grouping bits is simple mechanical work with no arithmetic. **Method 2 (through decimal)** involves more multiplication and division, so it is slower, but it is a useful double-check if you're unsure of your binary grouping.

Exercise

Convert $(144.4)_8$ to hexadecimal, using Method 1.

Solution

Step 1: $(144.4)_8 \rightarrow$ binary (3-bit groups): $001\ 100\ 100\ .\ 100 \rightarrow (1100100.100)_2$

Step 2: Regroup into 4 bits: $0110\ 0100\ .\ 100 \rightarrow$ pad $\rightarrow 0110\ 0100\ .\ 1000$

6 4 8

$$(144.4)_8 = (64.8)_{16}$$

This matches Chapter 8's earlier example directly, since $(1100100.10)_2 = (144.4)_8 = (64.8)_{16}$.

Binary Addition (and Addition in Other Bases)

12.1 Addition Rules with Carries

Binary addition follows the same logic as decimal addition (add column by column, carry when needed) but with only two possible digits.

For 2 bits

```
0 + 0 = 0 0    (0 with a 0 carry)
0 + 1 = 0 1    (1 with a 0 carry)
1 + 0 = 0 1    (1 with a 0 carry)
1 + 1 = 1 0    (0 with a 1 carry)
```

For 3 bits (when adding a carry-in too)

```
0+0+0 = 0 0    (0 with 0 carry)
0+0+1 = 0 1    (1 with 0 carry)
0+1+1 = 1 0    (0 with 1 carry)
1+1+1 = 1 1    (1 with 1 carry)
```

WHY DO WE NEED THE 3-BIT RULE?

When adding multi-digit binary numbers, a carry from the previous column often needs to be added too — so each column may need to add *three* bits (the two digits plus an incoming carry), not just two.

12.2 Adding Binary Numbers — Step by Step

Worked Example — $28 + 43 = 71$

```
  28 → 00011100
+ 43 → 00101011
-----
  71 → 01000111

Carries:  1 1 1 0
  00011100
+ 00101011
-----
  01000111
```

12.3 Adding Column by Column

From right to left, add each pair of digits. Write the sum digit, and carry any overflow to the next column on the left.

Worked Example

```
Carry:   0 1 1
         1 9 8
        + 2 6 4
        -----
Sum:      4 6 2
```

Exercise 3(a)

Add $(101101)_2$ with $(100111)_2$.

Solution 3(a)

```
Augend:   101101
Addend: + 100111
-----
Sum:      1010100
```

$(101101)_2 + (100111)_2 = (1010100)_2$

12.4 Addition of Base-r Numbers

The same column-by-column carry idea works for *any* base — the only difference is: whenever a column's sum reaches the base value (r) or more, you subtract r and carry 1 (instead of subtracting 10 like in decimal, or 2 like in binary).

Worked Example — Base 5 Addition Add $(34)_5 + (41)_5 + (24)_5$.

$$\begin{array}{r} 2 1 \qquad \leftarrow \text{carries} \\ (34)_5 \\ (41)_5 \\ + (24)_5 \\ \hline (2 0 4)_5 \end{array}$$

Rightmost column: $4+1+4 = 9$

$$9 \% 5 = 4 \quad (\text{write } 4)$$

$$9 / 5 = 1 \quad (\text{carry } 1)$$

Left column: $1+3+4+2 = 10$

$$10 \% 5 = 0 \quad (\text{write } 0)$$

$$10 / 5 = 2 \quad (\text{carry } 2, \text{ becomes the next digit})$$

Result: $(34)_5 + (41)_5 + (24)_5 = (204)_5$

EXAM TIP

The general column rule for any base r : compute the column total, then **write (total mod r)** and **carry $\lfloor \text{total} \div r \rfloor$** to the next column.

Exercise 2(a)

Add $(FF)_{16} + (F1)_{16}$.

Solution 2(a)

$$(FF)_{16} + (F1)_{16} = (1F0)_{16}$$

Exercise 2(b)

Add $(66)_7 + (55)_7$.

Solution 2(b)

$$(66)_7 + (55)_7 = (154)_7$$

Binary Subtraction (and Subtraction in Other Bases)

13.1 Subtraction Rules with Carries (Borrows)

```

0 - 0 = 0 0    (0 with a 0 carry/borrow)
1 - 1 = 0 0    (0 with a 0 carry/borrow)
1 - 0 = 0 1    (1 with a 0 carry/borrow)
0 - 1 = ?      (this needs a borrow from the next column!)

```

WHY IS 0 - 1 SPECIAL?

You cannot subtract 1 from 0 directly in a single column — just like in decimal, you must **borrow** from the next column to the left, which effectively turns the 0 into a 2 (in binary) before subtracting.

13.2 Subtracting Binary Numbers — Approach 1

Worked Example — $2 - 1 = 1$

```

  2  → 10
- 1  → 01
-----
  1  → 01

```

13.3 Subtracting Binary Numbers — Approach 2 (with borrowing)

Worked Example — $4 - 1 = 3$

$$\begin{array}{r} 1\ 1 \\ 4 \rightarrow 100 \\ - 1 \rightarrow 001 \\ \hline 3 \rightarrow 011 \end{array}$$

Here, borrowing happens across columns exactly as it would in decimal subtraction — you just borrow "2" instead of "10" because binary is base 2.

Exercise 5(b)

Subtract $(100111)_2$ from $(101101)_2$.

Solution 5(b)

$$\begin{array}{r} \text{Minuend:} \quad 101101 \\ \text{Subtrahend:} \quad - 100111 \\ \hline \text{Difference:} \quad 000110 \end{array}$$

$$(101101)_2 - (100111)_2 = (000110)_2$$

13.4 Subtraction of Base-r Numbers

The same borrowing idea extends to any base: when a digit in the minuend is smaller than the digit being subtracted, borrow 1 from the next column to the left — that borrowed 1 is worth "r" (the base) in the current column, not just 10.

Worked Example — Base 16 Subtraction

Compute $(4A6)_{16} - (1B3)_{16}$.

$$\begin{array}{r} 3\ 16 \quad \leftarrow \text{borrow notation} \\ (4\ A\ 6)_{16} \\ -(1\ B\ 3)_{16} \\ \hline (2\ F\ 3)_{16} \end{array}$$

In the middle column, A (10) is smaller than B (11), so we borrow: the borrowed 16 is added to A, giving $26 - 11 = 15 = F$. The leftmost 4 becomes 3 after lending the borrow.

Worked Example — Base 6 SubtractionCompute $(54)_6 - (35)_6$.

$$\begin{array}{r} \\ (5 \ 4)_6 \\ -(3 \ 5)_6 \\ \hline (1 \ 5)_6 \end{array}$$

Exercise 4(a)Compute $(71)_8 - (56)_8$.**Solution 4(a)**

$$(71)_8 - (56)_8 = (13)_8$$

Exercise 4(b)Compute $(21)_3 - (12)_3$.**Solution 4(b)**

$$(21)_3 - (12)_3 = (2)_3$$

Binary Multiplication (and Multiplication in Other Bases)

14.1 How Binary Multiplication Works

Binary multiplication follows the same procedure as decimal (long) multiplication — but because each digit is only 0 or 1, every partial product is either all zeros, or an exact copy of the multiplicand.

Table 14.1 — Single-bit multiplication rule

A	B	A × B
0	0	0
0	1	0
1	0	0
1	1	1

WHY DOES THIS MAKE BINARY MULTIPLICATION EASY?

Because binary multiplication can be seen as **repeated binary addition**: multiplying by 0 contributes a row of all zeros, and multiplying by 1 simply copies the multiplicand down (shifted to the correct column) — no actual multiplication arithmetic is needed, only shifting and addition (from Chapter 12).

14.2 Worked Example — 12×10 in Binary

12 in binary is 1100. 10 in binary is 1010.

Worked Example

```

  1 1 0 0
×  1 0 1 0
-----
  0 0 0 0
  1 1 0 0
  0 0 0 0
  1 1 0 0
-----
 1 1 1 1 0 0 0

```

So $(1100)_2 \times (1010)_2 = (1111000)_2$, which is $12 \times 10 = 120$ in decimal.

14.3 Extending to Multiple Digits

Worked Example — 1011×101

```
Multiplicand:      1011
Multiplier:        × 101
-----
Partial Products:  1011
                   0000-
                   1011--
-----
Product:           110111
```

Each row of the multiplier (reading right to left) either copies the multiplicand (if the multiplier bit is 1) or writes all zeros (if the bit is 0), shifted one column further left each time — then all partial products are added together using the binary addition rules from Chapter 12.

Exercise 6(a)

Multiply 1011 with 101.

Solution 6(a)

```
Multiplicand  1011
Multiplier    × 101
-----
Partial Prod. 1011
              0000-
              1011--
-----
Product       110111
```

14.4 Multiplication with Base-r (e.g. Hexadecimal)

The same long-multiplication layout works in any base — you just multiply and carry using that base's arithmetic.

Worked Example — Hexadecimal MultiplicationCompute $(2A3C)_{16} \times (B7)_{16}$.

```

  2A3C
×   B7
-----
 127A4      (this is 7 × 2A3C)
1D094      (this is B × 2A3C, shifted one column left)
-----
1E30E4

```

Working for the first row ($7 \times 2A3C$):

$7 \times C = 7 \times 12 = 84 \rightarrow$ write 4, carry 5
 $7 \times 3 + 5 = 26 \rightarrow$ write A, carry 1
 $7 \times A + 1 = 71 = 0 \times 47 \rightarrow$ write 7, carry 4
 $7 \times 2 + 4 = 18 = 0 \times 12 \rightarrow$ completes as 127A4

Working for the second row ($B \times 2A3C$), one column to the left:

$B \times C = 11 \times 12 = 132 \rightarrow$ write 4, carry 8
 $B \times 3 + 8 = 11 \times 3 + 8 = 41 \rightarrow$ write 9, carry 2
 $B \times A + 2 = 11 \times 10 + 2 = 112 \rightarrow$ write 0, carry 7
 $B \times 2 + 7 = 11 \times 2 + 7 = 29 = 1D \rightarrow$ completes as 1D094 (in the 16s column)

Adding the two partial products ($127A4 + 1D0940$):

```

  4+0      = 4
  A+4      = E
  7+9      = 16 = 0x10 → write 0, carry 1
  2+0+1    = 3
  1+D      = E
  -----
Result: 1E30E4

```

Exercise 7(a)Multiply $(34)_5$ with $(42)_5$.**Solution 7(a)**

$$(34)_5 \times (42)_5 = (3133)_5$$

Exercise 7(b)Multiply $(25)_9$ with $(36)_9$.

Solution 7(b)

$$(25)_9 \times (36)_9 = (1033)_9$$

Complements

This chapter is based entirely on the uploaded "Complements" document.

15.1 What Are Complements, and Why Do We Use Them?

A **complement** is a way of representing a number that makes certain operations — most importantly, **subtraction** — easier for digital circuits to perform, by turning subtraction into a form of addition.

WHY DO COMPLEMENTS EXIST?

Digital circuits are naturally good at addition. Using complements, a circuit can perform subtraction using only its addition hardware — this avoids needing a completely separate subtraction circuit.

15.2 Binary System Complements

Since the binary system has base $r = 2$, there are exactly **two** types of complements defined for it:

- **1's complement**
- **2's complement**

15.2.1 1's Complement

DEFINITION

The 1's complement of a binary number is found by changing every 1 to a 0, and every 0 to a 1. This is also simply called "taking the complement."

Worked Example Write the 1's complement of decimal 14.

Binary value of 14:	1110
1's complement of 14:	0001

COMMON MISTAKE

Only flipping some of the bits, or forgetting to flip leading zeros — every single bit must be flipped, including any leading zeros in the binary representation.

MEMORY TRICK

Think of 1's complement as a mirror: every bit swaps to its opposite. $1 \leftrightarrow 0$, $0 \leftrightarrow 1$.

15.2.2 2's Complement

DEFINITION / FORMULA

2's complement = 1's complement + 1

In words: the 2's complement of a binary number is obtained by adding 1 to the Least Significant Bit (LSB) of its 1's complement.

Worked Example — 15 Write the 2's complement of decimal 15.

```
Binary value of 15:      1111
1's complement of 15:    0000
Add 1:                   +   1
-----
2's complement of 15:    0001
```

Worked Example — 7 (showing why it works)

```
Binary value of 7:      111
2's complement of 7:  000 + 1 = 001
```

```
  0111  (+7)
+ 001   (2's complement of +7, i.e. -7)
-----
1 000   (result: 0, with an extra leading 1 that is discarded)
```

Adding +7 and its 2's complement (representing -7) produces 0 — exactly what we would expect from $7 + (-7) = 0$. This is precisely why 2's complement is so useful for subtraction: subtracting a number is the same as adding its 2's complement.

MEMORY TRICK

2's complement = "flip and add 1." Always do the 1's complement first, then add exactly 1 to the last bit.

COMMON MISTAKE

Forgetting to add the 1 after flipping the bits — that final step is what distinguishes 2's complement from 1's complement.

15.3 Decimal (Base-10) Complements

Just as binary (base 2) has two complements, the decimal system (base 10) also has two complements:

- **9's complement** (the base-10 equivalent of 1's complement)
- **10's complement** (the base-10 equivalent of 2's complement)

15.3.1 9's Complement

FORMULA

9's complement = $(r^n - 1) - N$

where N = the given number, n = number of digits in N , r = base (10 for decimal).

Worked Example Find the 9's complement of the decimal number 12349876.

Given: $N = 12349876$, $r = 10$, $n = 8$

$$\begin{aligned} 9's \text{ complement} &= (10^8 - 1) - 12349876 \\ &= 99999999 - 12349876 \\ &= 87650123 \end{aligned}$$

SHORTCUT

$(r^n - 1)$ is simply a string of n nines (e.g., 99999999 for $n = 8$). So the 9's complement can be found instantly by subtracting each digit of N from 9, digit by digit — no borrowing required!

15.3.2 10's Complement

FORMULA

10's complement = $r^n - N$

where N = the given number, n = number of digits in N , r = base (10 for decimal).

Worked Example Find the 10's complement of the decimal number 12349876.

Given: $N = 12349876$, $r = 10$, $n = 8$

$$\begin{aligned} 10's \text{ complement} &= 10^8 - 12349876 \\ &= 100000000 - 12349876 \\ &= 87650124 \end{aligned}$$

MEMORY TRICK

10's complement = 9's complement + 1 — notice 87650124 is exactly one more than 87650123 from the previous example. This mirrors the binary relationship: 2's complement = 1's complement + 1.

15.4 Summary Table of All Four Complements

Table 15.1 — Complement formulas at a glance

Complement	Base	Formula / Rule
1's complement	2	Flip every bit ($0 \leftrightarrow 1$)
2's complement	2	1's complement + 1
9's complement	10	$(10^n - 1) - N$
10's complement	10	$10^n - N$ (= 9's complement + 1)

Practice Note

The source material for this chapter (the uploaded Complements document) provides four fully worked examples rather than separate practice questions. Use the four worked examples above (1's complement of 14, 2's complement of 15, 2's complement of 7, and the 9's/10's complements of 12349876) as your practice set — cover the solution and try to reproduce each one from just the definition and formula before checking your answer.

Complete Cheat Sheet

Everything on these pages for last-minute revision — no new explanations, just quick-look references.

Powers of 2

$2^0=1$ $2^1=2$ $2^2=4$ $2^3=8$ $2^4=16$
 $2^5=32$ $2^6=64$ $2^7=128$ $2^8=256$ $2^9=512$
 $2^{10}=1024$

Base radix digit ranges

Base $r \rightarrow$ digits range from 0 to $(r-1)$.
Binary: 0–1 | Octal: 0–7 | Decimal: 0–9 | Hex: 0–F

Binary $\leftrightarrow$ Octal lookup (3-bit groups)

000=0 001=1 010=2 011=3
100=4 101=5 110=6 111=7

Binary $\leftrightarrow$ Hex lookup (4-bit groups)

0000=0 0001=1 0010=2 0011=3
0100=4 0101=5 0110=6 0111=7
1000=8 1001=9 1010=A 1011=B
1100=C 1101=D 1110=E 1111=F

Conversion formulas

Base-R $\rightarrow$ Decimal: $\sum(\text{digit} \times \text{base}^{\text{position}})$
Decimal $\rightarrow$ Base-R (whole): repeated division by R, read remainders bottom $\rightarrow$ top
Decimal $\rightarrow$ Base-R (fraction): repeated multiplication by R, read carries top $\rightarrow$ bottom

Complement formulas

1's complement = flip every bit
2's complement = 1's complement + 1
9's complement = $(10^n - 1) - N$
10's complement = $10^n - N$

Conversion flow (general bases)

Base-A $\rightarrow$ Decimal $\rightarrow$ Base-B (via positional formula, then repeated division/multiplication)
Binary $\leftrightarrow$ Octal $\rightarrow$ group/ungroup by 3 bits
Binary $\leftrightarrow$ Hex $\rightarrow$ group/ungroup by 4 bits
Octal $\leftrightarrow$ Hex $\rightarrow$ via binary (fast) or via decimal (slower, good check)

Addition / Subtraction / Multiplication in base r

Addition: column total $\rightarrow$ write (total mod r), carry $\lfloor \text{total} \div r \rfloor$
Subtraction: borrow = r from the next column when needed
Multiplication: same long-multiplication layout as decimal, using base- r arithmetic for each partial product

Things students commonly forget

- Position after the point starts at -1 , not 0
- Remainders read bottom $\rightarrow$ top; carries read top $\rightarrow$ bottom
- Always flip every bit for 1's complement, including leading zeros
- 2's complement needs the "+1" step — don't stop at 1's complement
- Hex remainders 10–15 must be written as A–F, never as two digits

Exam tips

- Always double-check a conversion by converting your answer back and comparing to the original.
- For octal/hex conversions, grouping through binary is usually fastest — use decimal as a backup check.
- Write place-value headers above digits before multiplying, to avoid position errors.